NAME: V Vishwa Teja ID NO: 2520090033

SKILL WEEK 12

1. To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill12.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Write the code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/resource.h>
#include <time.h>
#include <string.h>

int main()
{
    int pipefd[2];
    pid_t p1, p2;
    int status1, status2;
    struct timespec start, end;
    struct rusage usage;

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    clock_gettime(CLOCK_MONOTONIC, &start);

    p1 = fork();

    if (p1 == 0)
    {
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[0]);
        close(pipefd[1]);

        execlp("seq", "seq", "1", "100000", NULL);
        perror("seq");
        _exit(127);
    }

    if (p1 < 0)
    {
        perror("fork");
        close(pipefd[0]);
        close(pipefd[1]);
        return 1;
    }

    p2 = fork();

    if (p2 == 0)
    {
```

```
        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);
        close(pipefd[1]);

        execlp("wc", "wc", "-l", NULL);
        perror("wc");
        _exit(127);
    }

    if (p2 < 0)
    {
        perror("fork");
        close(pipefd[0]);
        close(pipefd[1]);
        waitpid(p1, NULL, 0);
        return 1;
    }

    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(p1, &status1, 0);
    wait4(p2, &status2, 0, &usage);

    clock_gettime(CLOCK_MONOTONIC, &end);

    double elapsed = (end.tv_sec - start.tv_sec) +
                     (end.tv_nsec - start.tv_nsec) / 1000000000.0;

    printf("\nPipeline Performance Report\n");
    printf("---------------------------\n");
    printf("Pipeline: seq 1 100000 | wc -l\n");
    printf("Execution time: %.6f seconds\n", elapsed);
    printf("Child user CPU time: %ld.%06ld seconds\n",
           (long)usage.ru_utime.tv_sec,
           (long)usage.ru_utime.tv_usec);
    printf("Child system CPU time: %ld.%06ld seconds\n",
           (long)usage.ru_stime.tv_sec,
           (long)usage.ru_stime.tv_usec);

    if (WIFEXITED(status1) && WEXITSTATUS(status1) == 0 &&
        WIFEXITED(status2) && WEXITSTATUS(status2) == 0)
        printf("Pipeline status: SUCCESS\n");
    else
        printf("Pipeline status: FAILURE\n");

    return 0;
}
```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill12.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc -Wall -Wextra skill12.1.c -o skill12.1
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill12.1
100000

Pipeline Performance Report
---------------------------
Pipeline: seq 1 100000 | wc -l
Execution time: 0.007846 seconds
Child user CPU time: 0.000000 seconds
Child system CPU time: 0.004721 seconds
Pipeline status: SUCCESS
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

2. To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill12.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Write the code:

```c
#include <stdio.h>

int main()
{
    printf("SYSTEM PROGRAMMING SHELL PROJECT\n");
    printf("================================\n\n");

    printf("1. ARCHITECTURE\n");
    printf("User Input -> Parser -> Process Manager -> Command Execution\n");
    printf("Process Manager -> Job Table -> Status Monitoring\n\n");

    printf("2. PIPELINE EXECUTION PATH\n");
    printf("Read command -> Parse pipe -> Create pipe -> Fork processes\n");
    printf("Redirect descriptors -> Execute commands -> Wait for children\n\n");

    printf("3. JOB CONTROL\n");
    printf("Create process group -> Register job -> Handle signals\n");
    printf("Update job state -> Resume, terminate, or complete job\n\n");

    printf("4. IMPORTANT SYSTEM CALLS\n");
    printf("fork()   : Creates a child process\n");
    printf("exec()   : Replaces process image with a command\n");
    printf("pipe()   : Creates a communication channel\n");
    printf("dup2()   : Redirects a file descriptor\n");
    printf("waitpid(): Waits for a child process\n");
    printf("kill()   : Sends a signal to a process or process group\n");
    printf("setpgid(): Sets a process group ID\n");
    printf("tcsetpgrp(): Changes the terminal foreground process group\n\n");

    printf("5. ERROR HANDLING\n");
    printf("Check system-call return values, report errors, and clean up\n");
    printf("file descriptors and child processes.\n\n");

    printf("6. DESIGN DECISIONS\n");
    printf("Use process groups for job control, pipes for communication,\n");
    printf("and waitpid() for child-process status monitoring.\n\n");

    printf("Documentation review completed.\n");

    return 0;
}
```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill12.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc -Wall -Wextra skill12.2.c -o skill12.2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill12.2
SYSTEM PROGRAMMING SHELL PROJECT
==============================

1. ARCHITECTURE
User Input -> Parser -> Process Manager -> Command Execution
Process Manager -> Job Table -> Status Monitoring

2. PIPELINE EXECUTION PATH
Read command -> Parse pipe -> Create pipe -> Fork processes
Redirect descriptors -> Execute commands -> Wait for children

3. JOB CONTROL
Create process group -> Register job -> Handle signals
Update job state -> Resume, terminate, or complete job

4. IMPORTANT SYSTEM CALLS
fork()    : Creates a child process
exec()    : Replaces process image with a command
pipe()    : Creates a communication channel
dup2()    : Redirects a file descriptor
waitpid(): Waits for a child process
kill()    : Sends a signal to a process or process group
setpgid(): Sets a process group ID
tcsetpgrp(): Changes the terminal foreground process group

5. ERROR HANDLING
Check system-call return values, report errors, and clean up
file descriptors and child processes.

6. DESIGN DECISIONS
Use process groups for job control, pipes for communication,
and waitpid() for child-process status monitoring.

Documentation review completed.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

3. To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill12.3.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Write the code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int main()
{
    const char *files[] = {
        "skill12.1.c",
        "skill12.2.c",
        "skill12.3.c",
        "skill12.4.c",
        "README.md",
        "architecture.md"
    };

    int total = sizeof(files) / sizeof(files[0]);
    int missing = 0;

    printf("PROJECT STRUCTURE VALIDATION\n");
    printf("============================\n");

    for (int i = 0; i < total; i++)
    {
        FILE *file = fopen(files[i], "r");

        if (file != NULL)
        {
            printf("[FOUND]   %s\n", files[i]);
            fclose(file);
        }
        else
        {
            printf("[MISSING] %s\n", files[i]);
            missing++;
        }
    }

    printf("\nTotal required files: %d\n", total);
    printf("Missing files: %d\n", missing);

    if (missing == 0)
    {
        printf("Validation result: PASS\n");
        return 0;
    }

    printf("Validation result: INCOMPLETE\n");
    return 1;
}
```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill12.3.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc -Wall -Wextra skill12.3.c -o skill12.3
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill12.3
PROJECT STRUCTURE VALIDATION
============================
[FOUND]   skill12.1.c
[FOUND]   skill12.2.c
[FOUND]   skill12.3.c
[MISSING] skill12.4.c
[MISSING] README.md
[MISSING] architecture.md

Total required files: 6
Missing files: 3
Validation result: INCOMPLETE
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

4. To Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill12.4.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Write the code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

void pipeline_demo()
{
    int pipefd[2];
    pid_t first, second;
    int status1, status2;

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return;
    }

    first = fork();

    if (first == 0)
    {
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[0]);
        close(pipefd[1]);

        execlp("seq", "seq", "1", "10", NULL);
        perror("seq");
        _exit(127);
    }

    if (first < 0)
    {
        perror("fork");
        close(pipefd[0]);
        close(pipefd[1]);
        return;
    }

    second = fork();

    if (second == 0)
    {
        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);
        close(pipefd[1]);

        execlp("wc", "wc", "-l", NULL);
        perror("wc");
```

```
        perror("wc");
        _exit(127);
    }

    if (second < 0)
    {
        perror("fork");
        close(pipefd[0]);
        close(pipefd[1]);
        waitpid(first, NULL, 0);
        return;
    }

    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(first, &status1, 0);
    waitpid(second, &status2, 0);

    if (WIFEXITED(status1) && WEXITSTATUS(status1) == 0 &&
        WIFEXITED(status2) && WEXITSTATUS(status2) == 0)
        printf("Pipeline demonstration: PASS\n");
    else
        printf("Pipeline demonstration: FAIL\n");
}

void signal_demo()
{
    int status;
    pid_t child = fork();

    if (child == 0)
    {
        execlp("sleep", "sleep", "30", NULL);
        perror("sleep");
        _exit(127);
    }

    if (child < 0)
    {
        perror("fork");
        return;
    }

    printf("Foreground demo process started: PID %d\n", child);

    sleep(2);

    printf("Sending SIGTERM to the demo process...\n");
```

```
    if (kill(child, SIGTERM) == -1)
    {
        perror("kill");
        waitpid(child, NULL, 0);
        return;
    }

    if (waitpid(child, &status, 0) == -1)
    {
        perror("waitpid");
        return;
    }

    if (WIFSIGNALED(status) && WTERMSIG(status) == SIGTERM)
        printf("Signal demonstration: PASS\n");
    else
        printf("Signal demonstration: FAIL\n");
}

int main()
{
    int choice;

    while (1)
    {
        printf("\nFINAL PROJECT DEMONSTRATION\n");
        printf("===========================\n");
        printf("1. Demonstrate pipeline\n");
        printf("2. Demonstrate signal handling\n");
        printf("3. Exit\n");
        printf("Enter choice: ");

        if (scanf("%d", &choice) != 1)
        {
            printf("Invalid input.\n");
            return 1;
        }

        switch (choice)
        {
            case 1:
                pipeline_demo();
                break;

            case 2:
                signal_demo();
                break;

            case 3:
```

```
            case 3:
                printf("Project review completed.\n");
                return 0;

            default:
                printf("Invalid choice. Try again.\n");
        }
    }
}
```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill12.4.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc -Wall -Wextra skill12.4.c -o skill12.4
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill12.4

FINAL PROJECT DEMONSTRATION
===========================
1. Demonstrate pipeline
2. Demonstrate signal handling
3. Exit
Enter choice: 1
10
Pipeline demonstration: PASS

FINAL PROJECT DEMONSTRATION
===========================
1. Demonstrate pipeline
2. Demonstrate signal handling
3. Exit
Enter choice: 2
Foreground demo process started: PID 919
Sending SIGTERM to the demo process...
Signal demonstration: PASS

FINAL PROJECT DEMONSTRATION
===========================
1. Demonstrate pipeline
2. Demonstrate signal handling
3. Exit
Enter choice: 3
Project review completed.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```